

AI Large Model Online Voice Assistant

AI Large Model Online Voice Assistant

1. Online Voice Configuration
2. Create a Startup Service (Systemd)
 - 2.1 Create a startup script
 - 2.2 Create the Systemd Service File
 - 2.3 Manage and Debug the Service

1. Online Voice Configuration

Before setting up auto-start, we must ensure that the program itself can work independently in an online state. This needs to be done by modifying the configuration file.

1. Open the configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify the configuration parameters:

Please check the following parameters in the file and ensure that their values are consistent with those shown below. If the parameters do not exist, please add them.

```
asr:                                # Speech node parameters
  ros__parameters:
    VAD_MODE: 2                     # VAD sensitivity
    sample_rate: 16000              # ASR recording audio sample rate
    frame_duration_ms: 30           # VAD frame size unit ms
    use_online_asr: True            # Whether to use online ASR
  recognition (True for online, False for offline)
  mic_serial_port: "/dev/ttyUSB0"  # Microphone serial port alias
  mic_index: 0                     # Microphone index
  language: 'en'                   # ASR language
  regional_setting : "international" # international:
  International version China: Domestic version

model_service:                      # Model server node parameters
  ros__parameters:
    language: 'en'                  # Large model interface language
    use_online_tts: True            # Whether to use online speech
  synthesis (True for online, False for offline)
  llm_platform: 'tongyi'           # Here is an example of Tongyi
```

- `use_online_asr` and `use_online_tts` must be set to `True`.
- `llm_platform` must be set to `tongyi`.

With this setting, everything will be online.

3. Save the file and recompile the project to apply the changes:

```
cd ~/yahboom_ws
colcon build
source install/setup.bash
```

After completing this step, the program is a purely online voice service.

2. Create a Startup Service (Systemd)

Now, we will create a `systemd` service to automatically run `largemodel_control.launch.py` when the system starts.

Users of Yahboom's AI large model image do not need to create it manually; just skip to the back and run the command to start on boot.

2.1 Create a startup script

To allow `systemd` to correctly load the ROS2 environment, the best practice is to create a simple `bash` script to encapsulate our startup command.

1. Create the script file:

In the directory (`~/yahboom_ws/src/largemodel/`), create a file named `start_largemodel.sh`.

```
vim ~/yahboom_ws/src/largemodel/start_largemodel.sh
```

2. Write the script content:

Copy and paste the following content into the script file.

```
#!/bin/bash

# Source the ROS2 Humble environment
source /opt/ros/humble/setup.bash

# Source the Yahboom workspace environment
source /home/sunrise/yahboom_ws/install/setup.bash

# Start the largemodel control script
ros2 launch largemodel largemodel_control.launch.py
```

Important note: Please make sure to replace `/home/sunrise/` in the script with your own user's home directory path.

3. Save and exit

4. Give the script execution permission:

```
chmod +x ~/yahboom_ws/src/largemodel/start_largemodel.sh
```

2.2 Create the Systemd Service File

1. Create the service file:

You need to use `sudo` permission to create this file.

```
sudo vim /etc/systemd/system/largemodel.service
```

2. Write the service configuration:

Copy and paste the following content completely into the service file.

```
[Unit]
Description=Robot Service
After=network.target sound.target graphical.target multi-user.target
Wants=network.target sound.target graphical.target multi-user.target

[Service]
Type=simple
User=sunrise
Group=sunrise
Environment=DISPLAY=:0
Environment=XDG_RUNTIME_DIR=/run/user/1000
Environment=PULSE_SERVER=unix:/run/user/1000/pulse/native
SupplementaryGroups=audio video
ExecStartPre=/bin/sleep 10
ExecStart=/home/sunrise/yahboom_ws/src/largemodel/start_largemodel.sh
Restart=on-failure
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

- `Environment=DISPLAY=:0` will output the GUI pop-up window to the desktop. If you are using an SSH connection, you can change it to `Environment=DISPLAY=:10`. You can query the specific parameters by entering `echo $DISPLAY` in the terminal.
- The path in `ExecStart` is exactly the same as the path of the startup script to be executed.

3. Save and exit.

2.3 Manage and Debug the Service

Now that your service has been created, we need to let `systemd` load it and set it to start on boot.

1. Reload the `systemd` daemon to let it read our newly created service file:

```
sudo systemctl daemon-reload
```

2. Set the service to start on boot:

```
sudo systemctl enable largemodel.service
```

3. Start the service immediately:

```
sudo systemctl start largemodel.service
```

4. Check the service status:

This is the most important command to verify that the service is running successfully.

```
sudo systemctl status largemodel.service
```

- * If you see `Active: active (running)`, congratulations, the service has started successfully!
- * If the status is `failed` or something else, please proceed to the next step for debugging.

5. View the service log (essential for debugging):

If the service fails to start, you can view all the real-time logs generated by the `ros2 launch` command through the following command, which is crucial for locating the problem.

```
journalctl -u largemodel.service -f
```

After completing all the above steps, the purely offline `largemodel` voice service will now be automatically started every time the machine is turned on.

When you need to turn off the auto-start service, you can run the following commands:

```
sudo systemctl stop largemodel.service  
sudo systemctl disable largemodel.service
```